A Trade Worth $11 That Reported $8,241

Problem. Crypto and fractional share trades imported with the wrong quantity. A 0.001379 BTC round trip reported a profit of $8,241.71. The real profit was $11.37.

Root cause. Three independent defects across three layers. The database column could not hold a fraction, the P&L function used `parseInt`, and five import paths rounded quantities up to 1.

Resolution. Fixed all three. Fixing any one alone would have replaced one wrong number with a different wrong number. The fix then exposed two further bugs that had been hiding each other.

Context

StrategyForge is a trading journal. Users connect a brokerage account through SnapTrade, an aggregator that handles the OAuth connection, and their trade history syncs automatically.

I had been working from SnapTrade's documentation to build a new import path. The docs tell you which fields can exist. They do not tell you what any particular broker actually sends. So I built an admin endpoint that returns the raw payload untouched, connected a real Robinhood account, and read what came back.

That is where I found this.

Step 1. Read the raw payload, not the docs

The diagnostic endpoint returns SnapTrade's JSON exactly as received, plus a count of which fields are actually populated. From the Robinhood Crypto account:

```
{
  "type": "SELL",
  "price": 80738.74,
  "units": -0.001379,
  "amount": 111.348,
  "symbol": { "symbol": "BTC" },
  "trade_date": "2026-05-12T01:09:09Z"
}
```

`units: -0.001379`. A fraction of a Bitcoin, which is completely normal. Robinhood supports fractional shares too, so the same pattern shows up on equities:

```
{
  "type": "SELL",
  "price": 258.04,
  "units": -0.572659,
  "symbol": { "symbol": "AAPL" }
}
```

Half a share of Apple.

Step 2. What the database could hold

```
quantity: integer("quantity").notNull(),
```

An integer. It cannot store 0.001379 at all.

Step 3. What the import did about that

```
quantity: Math.round(Math.abs(openPos.quantity)) || 1,
```

`Math.round(0.001379)` is `0`. Then `|| 1` catches the zero and substitutes 1.

So 0.001379 Bitcoin became 1 Bitcoin. Half a share of Apple became a whole share. The value was not rejected or flagged. It was quietly replaced with a plausible number.

I found five of these across the different import paths.

Step 4. Working out the damage

The BTC position was bought at 72,497.03 and sold at 80,738.74.

```
  80738.74   sell price
- 72497.03   buy price
──────────
   8241.71   per unit

x 0.001379   real quantity  =  $11.37   actual profit
x 1          stored quantity =  $8,241.71   reported profit
```

The journal was overstating that trade by a factor of about 725.

The Apple trade was less dramatic and still wrong. Bought at 176.3731, sold at 258.04, holding 0.572659 shares. Real profit $46.77. Reported $81.67.

This is worse than a crash. A crash tells you something is broken. This produced a confident, plausible, wrong number and put it on the user's dashboard.

Step 5. The second defect, one layer up

Changing the column to a decimal is the obvious fix. It is not sufficient.

```
const quantity = parseInt(trade.quantity || 0);
```

That is the P&L calculation. `parseInt("0.001379")` returns `0`.

So with the column fixed and nothing else, every fractional trade would have shown a profit of exactly zero. One wrong number swapped for another.

Step 6. Three layers, fixed together

1. **Column** changed to `decimal(18, 8)`. Eight decimal places covers Bitcoin down to a single satoshi.
2. **P&L math** changed from `parseInt` to `parseFloat`.
3. **Five import paths** now keep the real value instead of rounding it.

Step 7. What the column change broke

Drizzle returns decimal columns as strings, not numbers. That is a small detail with a wide blast radius, so I went looking for everything that touched the field rather than assuming three fixes were the whole job.

Eighteen places did arithmetic or comparison on it. Two of them mattered a lot.

Duplicate detection would have stopped working entirely.

```
if (existingTrade.quantity !== newTrade.quantity) {
  return false;
}
```

The stored value now arrives as `"5.00000000"` while an incoming trade carries the number `5`. A strict comparison says those are different. Every re-sync would have looked like brand new trades, and the journal would have filled with duplicates.

The edit form was already rejecting fractional trades.

```
quantity: z.number().min(1, "Quantity must be at least 1"),
```

A crypto trade could not be entered by hand at all. That was a pre-existing bug the investigation happened to walk into.

Both fixed. Also fixed: quantity sorting, which was sorting the values as text and putting "10" before "9", and the CSV export, which multiplied by the raw string.

Step 8. Testing the fix, and failing

I wrote unit tests using the real BTC and Apple values from the payload, and they passed. Then I tried entering `1.25` in the form on staging and got this from the browser:

> *Please enter a valid value. The two nearest valid values are 1 and 2.*

That is not our validation. That is the browser rejecting the input before any of our code runs, because `type="number"` with no `step` attribute defaults to whole numbers only.

Behind it was a third problem in the same field:

```
onChange={(e) => field.onChange(parseInt(e.target.value) || 0)}
```

Another `parseInt`, in the change handler. My earlier search had looked for `parseInt(...quantity...)` and this one reads `parseInt(e.target.value)`, so it did not match.

Both fixed. Test passed on the retry.

Step 9. A regression I caused

While checking the form I found something worse than the thing I was checking.

Making the column a decimal meant the validation schema now expected a string. Five endpoints validate through that schema, including manual trade creation. All five would have rejected a numeric quantity with "Expected string, received number."

Manual trade creation would have failed outright. Nobody had seen it because the browser's step validation was blocking submission first. The two bugs were concealing each other.

I fixed it at the schema rather than at five call sites, so a future path cannot miss it. It now accepts either a number or a string, normalises to eight decimal places, and rejects zero, negatives and anything unparseable.

Step 10. Checking the other import route

The API is not the only way trades get in. Users also upload broker CSVs, so I went through that path too.

The ThinkOrSwim parser had the same defect in a different disguise:

```
const qty = Math.abs(parseInt(row['Qty']) || 1);
```

Same outcome. A half share becomes a whole one.

There is also an AI-assisted parser for unrecognised CSV formats. Its instructions described quantity with only whole-number examples, which invites the model to round. I rewrote that section to state plainly that fractional values must never be rounded, and added real BTC and Apple examples to the sample output.

What I'd take from this

Documentation tells you what can exist, not what arrives. SnapTrade's field definitions are accurate and complete. They still could not have told me that Robinhood sends fractional units on almost every crypto row. Only the payload could.

`|| 1` is not a safety net. It looks like defensive code. What it actually does is convert a value the system could not represent into one that looks reasonable. A hard failure would have surfaced this years earlier.

Check the blast radius before declaring a fix done. The column change was correct and broke eighteen other places. Two of them would have been worse than the original bug. The search for those took longer than the fix and was worth more.

Two bugs can hide each other. The browser validation and the schema regression were each masking the other. If I had only tested the form, I would have found one and shipped the other.

Grep for the pattern, not the variable name. My first sweep searched for `parseInt` next to `quantity` and missed `parseInt(e.target.value)` in a field that was clearly the quantity input. The second sweep looked for the pattern instead and found it.

Figures are from a real broker sync on a test account. Account identifiers removed.